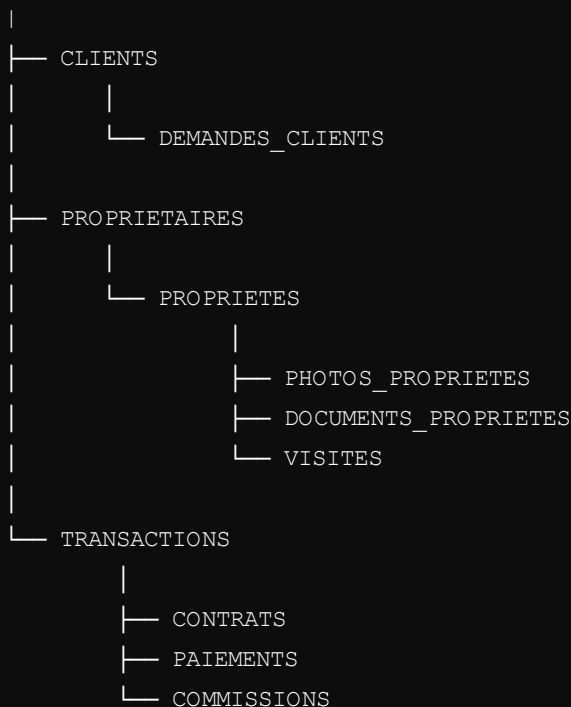


Pour une agence immobilière comme **Ciento Immobilier**, je recommande une base de données conçue autour de 6 modules principaux :

1. Gestion des utilisateurs
2. Gestion des clients
3. Gestion des propriétaires
4. Gestion des biens immobiliers
5. Gestion des transactions
6. Gestion documentaire

Schéma général

UTILISATEURS



1. Table utilisateurs

Permet de gérer les employés.

SQL

```
CREATE TABLE utilisateurs (
  id SERIAL PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100),
  email VARCHAR(150) UNIQUE,
  telephone VARCHAR(30),
```

```
mot_de_passe_hash TEXT NOT NULL,  
role VARCHAR(50) NOT NULL,  
actif BOOLEAN DEFAULT TRUE,  
date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Rôles

```
Administrateur  
Directeur  
Agent immobilier  
Assistant  
Comptable
```



2. Table clients

SQL



```
CREATE TABLE clients (  
    id SERIAL PRIMARY KEY,  
    code_client VARCHAR(20) UNIQUE,  
    nom VARCHAR(100) NOT NULL,  
    prenom VARCHAR(100),  
    telephone VARCHAR(30),  
    telephone_secondaire VARCHAR(30),  
    email VARCHAR(150),  
    adresse TEXT,  
    ville VARCHAR(100),  
    profession VARCHAR(100),  
    budget_min NUMERIC(15,2),  
    budget_max NUMERIC(15,2),  
    source_client VARCHAR(100),  
    observations TEXT,  
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. Table demandes_clients

Critères de recherche du client.

SQL



```
CREATE TABLE demandes_clients (  
    id SERIAL PRIMARY KEY,
```

```
client_id INTEGER REFERENCES clients(id),
type_operation VARCHAR(20),
type_bien VARCHAR(50),
zone_recherche TEXT,
chambres INTEGER,
salles_bain INTEGER,
budget NUMERIC(15,2),
statut VARCHAR(30),
date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4. Table propriétaires

SQL



```
CREATE TABLE proprietaires (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(100),
    prenom VARCHAR(100),
    telephone VARCHAR(30),
    email VARCHAR(150),
    adresse TEXT,
    numero_identite VARCHAR(50),
    observations TEXT,
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5. Table propriétés

Cœur du système.

SQL



```
CREATE TABLE proprietes (
    id SERIAL PRIMARY KEY,

    reference_bien VARCHAR(30) UNIQUE,

    proprietaire_id INTEGER
        REFERENCES proprietaires(id),

    titre VARCHAR(255),

    description TEXT,

    type_bien VARCHAR(50),
```

```
type_operation VARCHAR(20),

adresse TEXT,

ville VARCHAR(100),

quartier VARCHAR(100),

latitude NUMERIC(10,7),

longitude NUMERIC(10,7),

superficie NUMERIC(12,2),

nombre_chambres INTEGER,

nombre_salles_bain INTEGER,

nombre_garages INTEGER,

prix NUMERIC(15,2),

devise VARCHAR(10),

statut VARCHAR(30),

date_ajout TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Type bien

Maison
Appartement
Terrain
Villa
Bureau
Local commercial
Entrepôt
Immeuble



Statut

Disponible
Réservé
Vendu
Loué
Suspendu



6. Table caractéristiques

Pour ajouter des équipements.

SQL



```
CREATE TABLE caracteristiques (  
    id SERIAL PRIMARY KEY,  
    nom VARCHAR(100)  
);
```

Exemples :

Piscine
Génératrice
Jardin
Puits
Climatisation
Caméra
Internet



7. Table liaison caractéristiques

SQL



```
CREATE TABLE proprietes_caracteristiques (  
    propriete_id INTEGER REFERENCES proprietes(id),  
    caracteristique_id INTEGER REFERENCES caracteristiques(id),  
    PRIMARY KEY(propriete_id, caracteristique_id)  
);
```

8. Table photos

SQL



```
CREATE TABLE photos_proprietes (  
    id SERIAL PRIMARY KEY,  
  
    propriete_id INTEGER  
        REFERENCES proprietes(id),  
  
    chemin_fichier TEXT,
```

```
photo_principale BOOLEAN DEFAULT FALSE,  
  
date_upload TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

9. Table documents

</> SQL



```
CREATE TABLE documents_proprietes (  
    id SERIAL PRIMARY KEY,  
  
    propriete_id INTEGER  
        REFERENCES proprietes(id),  
  
    nom_document VARCHAR(255),  
  
    type_document VARCHAR(100),  
  
    chemin_fichier TEXT,  
  
    date_upload TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

10. Table visites

</> SQL



```
CREATE TABLE visites (  
    id SERIAL PRIMARY KEY,  
  
    client_id INTEGER REFERENCES clients(id),  
  
    propriete_id INTEGER REFERENCES proprietes(id),  
  
    agent_id INTEGER REFERENCES utilisateurs(id),  
  
    date_visite TIMESTAMP,  
  
    compte_rendu TEXT,  
  
    statut VARCHAR(30)  
);
```

11. Table transactions

SQL



```
CREATE TABLE transactions (  
    id SERIAL PRIMARY KEY,  
  
    reference_transaction VARCHAR(50) UNIQUE,  
  
    client_id INTEGER REFERENCES clients(id),  
  
    propriete_id INTEGER REFERENCES proprietes(id),  
  
    agent_id INTEGER REFERENCES utilisateurs(id),  
  
    type_transaction VARCHAR(20),  
  
    montant NUMERIC(15,2),  
  
    date_transaction DATE,  
  
    statut VARCHAR(30),  
  
    observations TEXT  
);
```

12. Table contrats

SQL



```
CREATE TABLE contrats (  
    id SERIAL PRIMARY KEY,  
  
    transaction_id INTEGER REFERENCES transactions(id),  
  
    numero_contrat VARCHAR(50),  
  
    date_signature DATE,  
  
    date_debut DATE,  
  
    date_fin DATE,  
  
    fichier_pdf TEXT  
);
```

13. Table paiements

</> SQL



```
CREATE TABLE paiements (  
    id SERIAL PRIMARY KEY,  
  
    transaction_id INTEGER REFERENCES transactions(id),  
  
    montant NUMERIC(15,2),  
  
    mode_paiement VARCHAR(50),  
  
    reference_paiement VARCHAR(100),  
  
    date_paiement DATE,  
  
    statut VARCHAR(30)  
);
```

14. Table commissions

</> SQL



```
CREATE TABLE commissions (  
    id SERIAL PRIMARY KEY,  
  
    transaction_id INTEGER REFERENCES transactions(id),  
  
    agent_id INTEGER REFERENCES utilisateurs(id),  
  
    pourcentage NUMERIC(5,2),  
  
    montant NUMERIC(15,2),  
  
    date_calcul DATE  
);
```

15. Journal d'activité

Très utile pour la sécurité.

</> SQL



```
CREATE TABLE journal_activites (  
    id SERIAL PRIMARY KEY,  
  
    utilisateur_id INTEGER REFERENCES utilisateurs(id),  
  
    action VARCHAR(255),  
  
    table_concernee VARCHAR(100),  
  
    enregistrement_id INTEGER,  
  
    date_action TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Fonctionnalités avancées que je recommande dès le départ

- Authentification sécurisée (bcrypt)
- Gestion des rôles et permissions
- Sauvegarde PostgreSQL automatique
- Génération PDF (contrats, reçus)
- Import/export Excel
- Historique des modifications
- Tableau de bord avec statistiques
- Gestion multi-agences (si Ciento Immobilier ouvre plusieurs bureaux)

Cette architecture est suffisamment robuste pour gérer **plusieurs dizaines de milliers de clients et biens immobiliers** avec Flask, SQLAlchemy et PostgreSQL.